MEMORIA TÉCNICA

Práctica 1

Programación para las Comunicaciones

Boryslav Tkachuk

Índice

1. Introducción 1
2. Funcionalidades 1
   2.1. Funcionalidades obligatorias 1
   2.2 Funcionalidades opcionales 2
   2.3. Seguridad. Certificados X.509 2
3. Implementación 3
   3.1. Protocolo y el formato de las tramas 3
   3.2. Modelo de E/S 4
   3.3. Hilos 4
   3.4. Organización del código 4
4. Ejecución 5
5. Autoría 5

1. Introducción

En esta memoria describimos la práctica 1 de Programación para las Comunicaciones. La cual es un servidor HTTP/HTTPS multihilo y un cliente de consola, ambos escritos en Java sobre sockets. El protocolo HTTP/1.1, líneas de petición y respuesta, cabeceras, cuerpo y cookies está implementado a mano.

La aplicación consiste en un carrito de la compra, en cual el contenido del carrito viaja en una cookie que el servidor valida contra un catálogo de productos cargado desde un fichero. El servidor escucha en dos puertos a la vez, el 10000 para HTTP y el 20000 para HTTPS, y atiende cada conexión en un hilo propio. En HTTPS se usa TLS 1.2 con autenticación mutua, el servidor y el cliente presentan certificados X.509 firmados por una misma autoridad certificadora creada para la práctica.

Además de la funcionalidad obligatoria, se han añadido varias funcionalidades opciones como son vaciar el carrito, modificar la cantidad de un producto, un límite de 99 unidades por producto y la persistencia de las cookies del cliente entre ejecuciones. La memoria recoge las funcionalidades, la implementación, la forma de ejecutar el proyecto y la declaración de autoría.

2. Funcionalidades

2.1. Funcionalidades obligatorias

El servidor atiende peticiones HTTP/1.1 en el puerto 10000 y HTTPS en el 20000, cada una en un hilo propio. El cliente es una aplicación de consola que muestra la respuesta HTTP completa. Se ofrecen las siguientes operaciones:

- Consultar el inicio y el catálogo
- Consultar un producto
- Consultar el carrito
- Añadir un producto
- Eliminar un producto
- Cambiar entre HTTP y HTTPS

El carrito no se guarda en el servidor, sino que viaja en la cookie carrito, con el formato `id:cantidad|id:cantidad`. En cada petición el servidor la lee, la valida contra el catálogo (cargado desde `catalogo.csv`), aplica la operación y devuelve la cookie actualizada mediante Set-Cookie. En HTTPS la autenticación es mutua, es decir, el servidor exige certificado de cliente y ambos certificados están firmados por la misma CA.

2.2 Funcionalidades opcionales

Las aportaciones realizadas son las siguientes:

- Vaciar el carrito: POST /carrito/vaciar borra la cookie enviando una fecha Expires pasada.
- Modificar la cantidad: POST /carrito/modificar fija la cantidad de un producto, de 0 a 99. Con 0 lo elimina.
- Límite de 99 unidades: se aplica por producto. Al alcanzarlo, la respuesta lo indica y el carrito no cambia.
- Validación de la cookie: el servidor trata la cookie como dato no fiable, es decir, descarta identificadores que no están en el catálogo y cantidades no válidas o fuera de rango.
- Cookies persistentes en el cliente: el cliente guarda las cookies en cookies.txt y las recupera al arrancar, así el estado del carrito se mantiene.
- Códigos de estado: además del 200, se devuelven 400 (cantidad no válida), 404 (ruta o producto desconocido) y 405 (método no permitido).

2.3. Seguridad. Certificados X.509

La CA se creó con `openssl req -x509` (RSA de 2048 bits, válida 10 años). Para el servidor y el cliente se generó un par de claves con `keytool -genkey` y una petición con `keytool -certreq`. La CA firmó cada petición con `openssl ca`, y los certificados firmados se importaron en los almacenes PKCS12, primero la CA y después el certificado propio. El servidor usa `server.pfx` y el cliente `cliente.pfx`. Cada parte confía únicamente en `cacert.pem`, y el servidor exige certificado de cliente con `setNeedClientAuth(true)`.

Certificado	DN completo	Emisor
CA	CN=ca, OU=ppc, O=umu, C=es	Autofirmado (CN=ca)
Servidor	CN=localhost, OU=ppc, O=umu, C=es	CN=ca, OU=ppc, O=umu, C=es
Cliente	CN=Boryslav Tkachuk, OU=ppc, O=umu, C=es	CN=ca, OU=ppc, O=umu, C=es

3. Implementación

3.1. Protocolo y el formato de las tramas

La comunicación sigue HTTP/1.1 sobre TCP, en el puerto 10000 o sobre TLS 1.2, en el puerto 20000. Cada petición usa una conexión nueva y el servidor la cierra al responder. Las líneas terminan en CRLF y una línea vacía separa las cabeceras del cuerpo. La longitud del cuerpo se indica con `Content-Length`, en bytes.

Petición (ejemplo: añadir un producto):

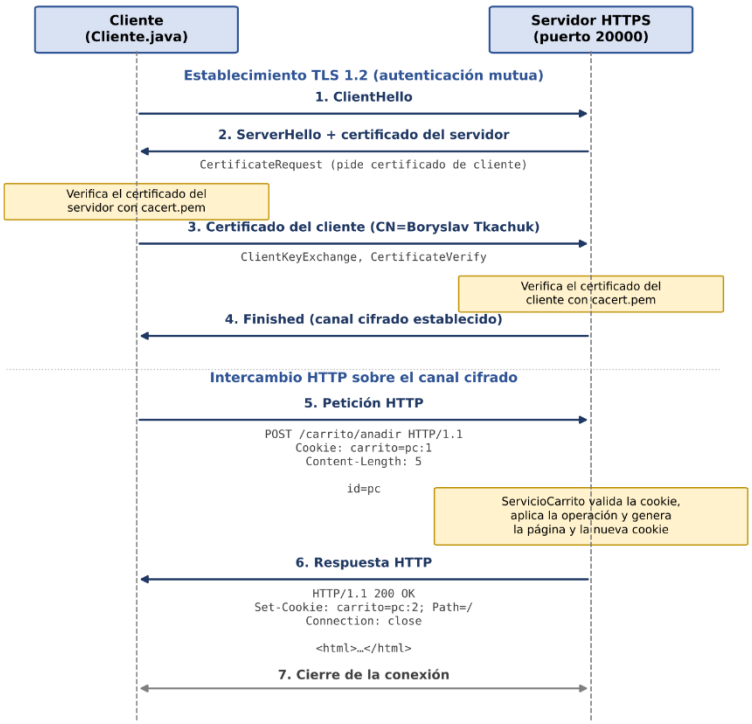
```
POST /carrito/anadir HTTP/1.1
Host: localhost:10000
Cookie: carrito=pc:1
Content-Length: 5

id=pc
```

Respuesta:

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: <bytes del cuerpo>
Date: <fecha>
Server: <nombre>
Connection: close
Set-Cookie: carrito=pc:2; Path=/

<html> ... </html>
```



El diagrama de secuencia muestra el intercambio en HTTPS, handshake TLS con certificados en ambos sentidos, petición, respuesta y cierre de la conexión.

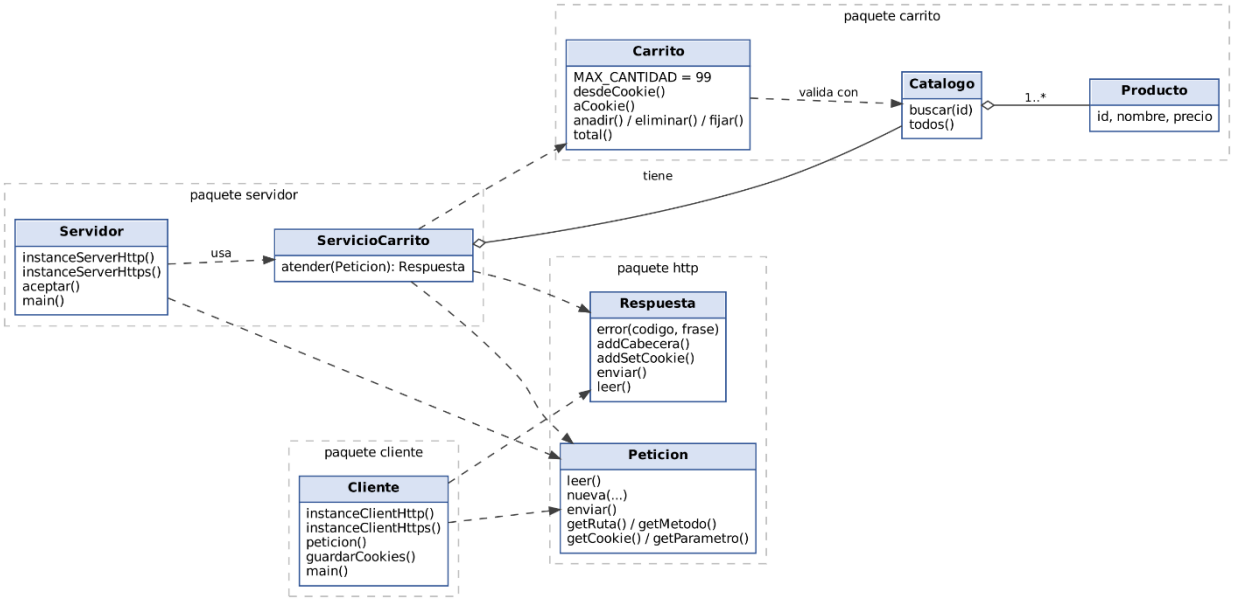
3.2. Modelo de E/S

Se usa E/S bloqueante de `java.net` y `javax.net.ssl` (`ServerSocket`, `Socket`, `SSLServerSocket`, `SSLSocket`). Cada lectura espera hasta recibir datos. Para que una conexión inactiva no bloquee un hilo indefinidamente, todos los sockets tienen `setSoTimeout(10000)`, pasados 10 segundos sin datos, la lectura falla y se cierra la conexión. La cabecera se lee línea a línea y el cuerpo se lee según el valor de `Content-Length`.

3.3. Hilos

- **Servidor:** El hilo main crea dos hilos de escucha, uno por puerto (HTTP y HTTPS). Cada uno acepta conexiones en bucle y, por cada conexión, crea un hilo nuevo que lee la petición, la atiende con `ServicioCarrito`, responde y cierra el socket. El servidor no comparte estado entre peticiones, porque el carrito viaja en la cookie. Por eso no hace falta sincronización.
- **Cliente:** Un único hilo (main) que muestra el menú y realiza cada petición de forma síncrona.

3.4. Organización del código



Paquete	Clase	Responsabilidad
servidor	Servidor	Crea los sockets HTTP y HTTPS (con el contexto SSL), acepta conexiones y lanza un hilo por conexión
servidor	ServicioCarrito	Enruta cada ruta y método, aplica la operación sobre el carrito y genera la página y la cookie
http	Peticion	Lee y analiza la petición y permite construirla y enviarla
http	Respuesta	Construye, envía y lee respuestas
carrito	Producto	Producto del catálogo
carrito	Catalogo	Carga catalogo.csv y busca productos por id
carrito	Carrito	Contenido del carrito, serialización a cookie y validación contra el catálogo
cliente	Cliente	Menú de consola, conexión HTTP/HTTPS, gestión y persistencia de cookies y muestra de la respuesta.

4. Ejecución

El proyecto tiene dos clases main: `servidor.Servidor` y `cliente.Cliente`. Las dos deben ejecutarse con el directorio de trabajo en la raíz del proyecto, ya que leen `src/catalogo.csv` y los almacenes de `cert/` con rutas relativas.

Servidor

```
java -cp bin servidor.Servidor
```

Arranca dos hilos de escucha: HTTP en el puerto 10000 y HTTPS en el 20000. Usa `cert/server.pfx` y confía en `cert/cacert.pem`.

Cliente

```
Java -cp bin cliente.Cliente
```

Muestra un menú de consola que se conecta a localhost. La opción 6 alterna entre HTTP y HTTPS. Para HTTPS usa `cert/cliente.pfx` y confía en `cert/cacert.pem`. Las cookies se guardan en `cookies.txt`, en el directorio de trabajo.

Desde Eclipse: el directorio de trabajo por defecto es la raíz del proyecto. Se ejecuta `Run As → Java Application` sobre Servidor y después sobre Cliente. Antes de volver a lanzar el servidor hay que detener la instancia anterior, porque los puertos quedan ocupados.

Las contraseñas no se piden por teclado, están constantes en el código. La clave privada de la CA (`cert/cakey.pem`) solo se necesita para firmar certificados nuevos, y su contraseña es `cakey`.

Se puede probar el funcionamiento desde un navegador, al abrir `https://localhost:20000/` con el certificado de cliente instalado y la CA como autoridad de confianza.

5. Autoría

El servidor, el cliente y los certificados los he desarrollado y probado yo, y entiendo cualquier parte del código entregado.

He utilizado la IA como asistente en estos casos:

- Revisión y diseño: Me ayudó con la estructura de paquetes y comentó el código.
- Depuración: Me ayudó a localizar errores (como fallos por typos, erratas, puertos ocupados) y proponerme casos de prueba para comprobar si el código era robusto.
- Certificados: Me guió en la ejecución de los comandos para crear la CA.
- Código: Me ayudó a escribir HTML.
- Diagramas.